

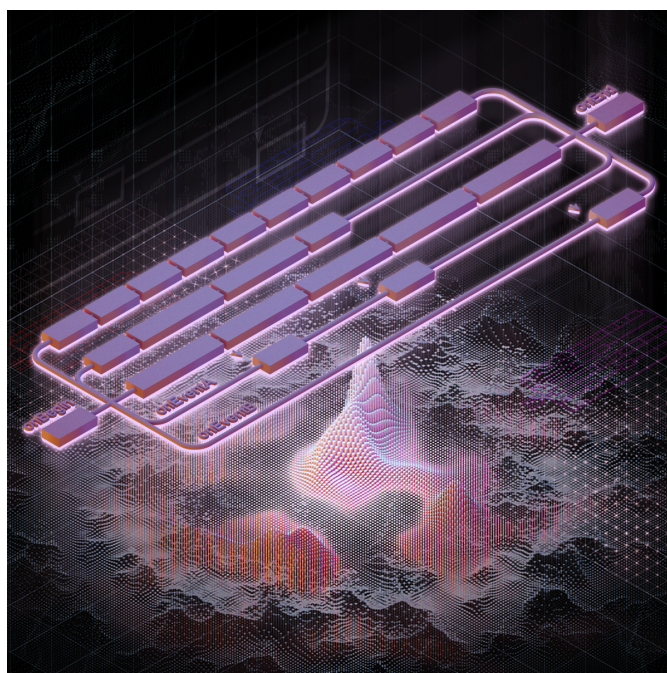
Программирование игрового процесса

На примере игры The Last of Us

Валерия Пудова

valery.hww@gmail.com

18.05.2022



Abstract

В документе кратко изложена базовая система игры *Game Runtime Base Foundation System*, используемая в серии игр “The Last of Us”. Игры компании *Naughty Dog* - это хороший пример сочетания принципов достаточности *KIS* и гибкости *Customization*.

Информация в файле - результат беглого *reverse engineering*, поэтому может содержать не полную и не точную информацию, но ее, в целом, достаточно чтобы понять суть предложенного метода.

Keywords Object System, Runtime, Scripting, Finite State Machines

Отказ от ответственности

Я не работаю в *Naughty Dog*, и не владею секретами компании или секретами *Last Of Us*, я делюсь теми своими знаниями которые получила сама, исследуя саму игру и читая информацию в интернете и книгах. Фрагменты приведенного кода доступны в различных источниках на которые я ссылаюсь в конце документа.

Disclaimer

I don't work at *Naughty Dog*, nor do I have any secret knowledge of *The Last of Us*, except what I figured out myself from the disc. So a lot of this may well be wrong. Take it with a pinch of salt. Most of the code samples in this document are taken from the sources listed at the end of the document.

Содержание

1	Введение	5
2	Управление ассетами	6
3	Редактор мира	7
4	Объектная система игры	8
5	Spawning	10
6	Сигналы и сообщения	11
7	State Update	12
8	Идентификаторы объектов	14
9	Синтаксис DC	16
10	Анимационные состояния	18
11	Конечные автоматы	20
12	Объявление переменных состояния	20
13	Параллельное исполнение процессов	20
14	Интеграция VM в Engine	23
15	DC компилятор	24
16	Формат SID-файла	25
17	Формат DCI-файла	26
18	Формат бинарного DC-файла	27
19	Виртуальная машина	28
20	Система команд VM	29
20.1	Использование регистров и констант	33
21	Результат	35
22	В завершение	36

1 Введение

В компании *Naughty Dog (ND)* используется набор инструментов разработчика, состоящий из множества различных утилит. Некоторые эти утилиты имеют собственный *GUI*, тогда как другие остаются в виде команд для консоли [Gre14]. Такой подход имеет определенные преимущества в сравнении с интегрированными средами разработки (*IDE*) игр, таких как Unity 3D.

- Требуется меньше затрат на разработку *GUI*, поэтому больше ресурсов можно потратить на саму игру
- Проще организовать совместную работу команды в целом
- Утилиты по отдельности требуют меньше ресурсов *ПК*. И поэтому они более удобны для выполнения конкретной задачи: *World, FX, Shaders*, редакторы и т.д.
- С таким подходом легче организовать автоматизацию производства
- Инструменты работают не деструктивно: если дизайнер ничего не менял, то и проект остается неизменным

Основная особенность *ND* подхода основана на использовании анимаций и динамическом обновлении окружения среды исполнения — не требуется перекомпиляция и перезапуск проекта чтобы увидеть результат.

2 Управление ассетами

Система управления *ассетами* — предназначена для организации совместной работы над проектом, отслеживания различных версий исходных файлов¹, устранения конфликтных ситуаций, а также для генерации актуального состояния всех медиа ресурсов проекта и сохранения результата в оптимальной для поковой загрузки форме. Эта система основана на распределенной базе данных, которая реализована с использованием СУБД или *xml* файлов². База данных содержит информацию о всех входных медиа-ресурсах и позволяет создать папку ассетов проекта и наполнить её контентом. Каждая запись содержит не только параметры импорта, но и дополнительную информацию, например, комментарии. Для разрешения конфликтов для каждого файла сохраняется история действий над ним: переименование, перемещение, удаление и т.д.

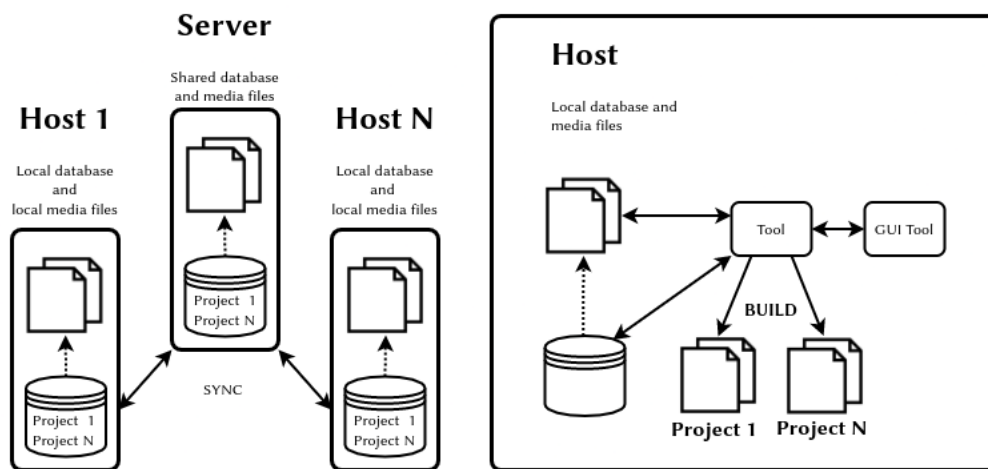


Рис. 1: Управление ассетами

Не стоит пренебрегать системой управления ассетами: от нее зависит очень много, так как создание игр — это в большей степени работа с ассетами, нежели программирование.

¹Таких как .psd, .tga, .mb, .fbx и т.д.

²Или комбинации обоих подходов

3 Редактор мира

Редактор никак не связан с самим движком и это является преимуществом³. Он использует текстовые конфигурационные файлы *schema*. Каждый файл содержит описание параметров для одного типа объектов сцены. Для каждого поля декларируются тип, значение по умолчанию, минимальное и максимальное значения, возможные значения для перечисляемых типов. Поставив пустой объект на сцену, дизайнер выбирает какую схему использует этот объект. После этого дизайнер может заполнять поля, так как список полей описан в файле схемы. Дизайнер имеет возможность самостоятельно добавлять или удалять поля в файл *schema*. В конечном итоге, редактор мира запишет структуру всей сцены, например, в виде *xml* или другого формата файла. Подход в целом иллюстрирован на рисунке 2

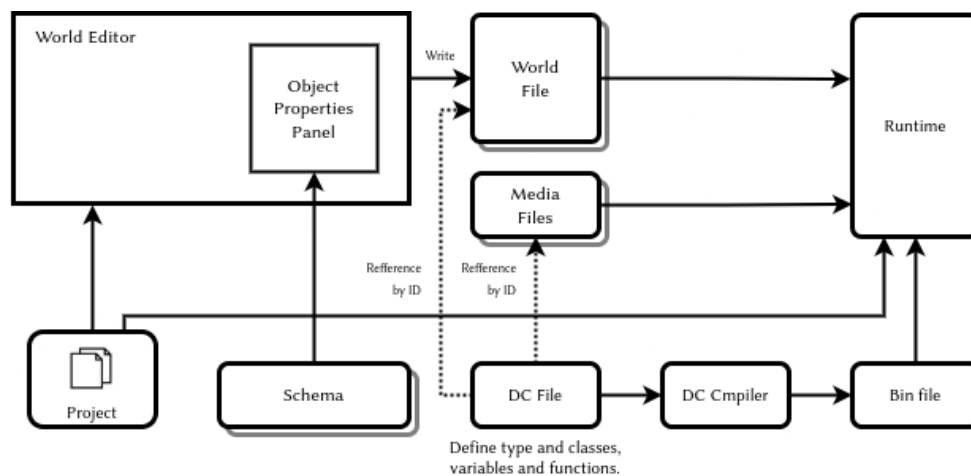


Рис. 2: Редактор мира и файл *schema*

Существуют следующие типы [Gre17] объектов сцены:

- *Spawner* — Процесс, производящий инстанцирование персонажа
- *Spline* — 3D кривая
- *Region* — Фрагмент мира
- *Nav Mesh* — Навигационная геометрия (сетка навигации)
- *Static Background Geometry* — геометрия статического фона

В параметрах объекта могут присутствовать такие поля как *archetype*, а иногда и *parent-archetype*. Эти параметры определяют то, какой объект на самом деле будет использован, какие компоненты он имеет и какие у них поля.

³об этом сказано во введении

4 Объектная система игры

Иерархия классов изображена на рисунке ниже. Она не имеет глубокой вложенности и построена от одного класса-предка. Теоретически система могла быть реализована как динамическая компонентная система[Coh10] или другом варианте *Data Oriented* программирования. Приблизительная структура классов изображена на диаграмме

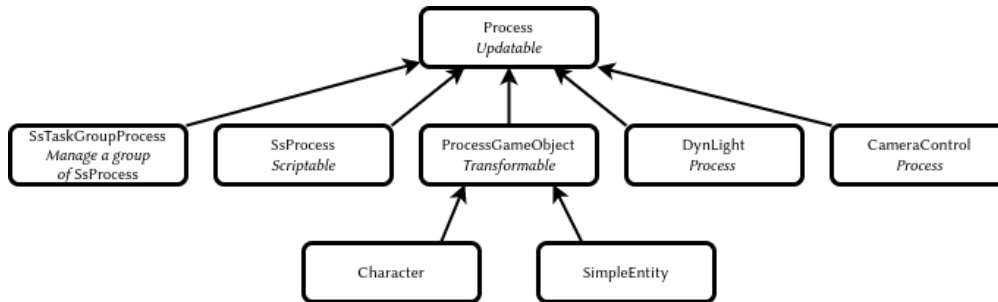


Рис. 3: Структура классов низкого уровня [Gre06]

Базовый класс всех персонажей *ProcessGameObject* и классы от него унаследованные, представляют собой хост-объект, функционал которого расширяется с помощью композиции, смотри рисунок 4. При использовании *MVC* можно создать два отдельных класса, каждый со своим набором параметров состояния и со своим набором компонентов.

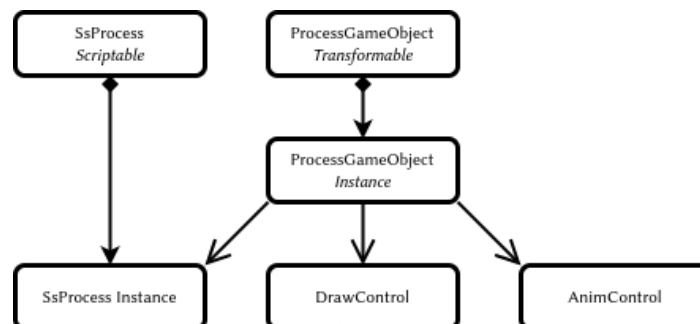


Рис. 4: Композиция игровых объектов [Gre06]

Объектная система игры основана на *LISP*-подобном языке, на котором описываются структуры, классы, экземпляры классов вместе с хранимыми в них данными, а также переменные и функции. Этот файл компилируется в файлы *.h*, *.bin* и *.dc*. Кроме всего перечисленного, *.dc*-файл содержит код машин состояний игровых объектов. Этот код организован как множество параллельных процессов, работающих в условиях кооперативной многозадачности. У процессов имеется механизм синхронизации, основанный на сигналах.

- *.h* — предназначен для сборки *C* компилятором и обеспечивает прямое использование динамических данных бинарным кодом
- *.bin* — содержит только данные структур и функции. Загружается игрой в среду исполнения
- *.dci* — текстовый файл с декларацией *import*-файлов и *export*-дефиниций

Изменение *DC*-файла требует перекомпиляции проекта только если изменилась структура, то есть был изменен *.h*-файл. Принцип работы системы изображен на рисунке 5.

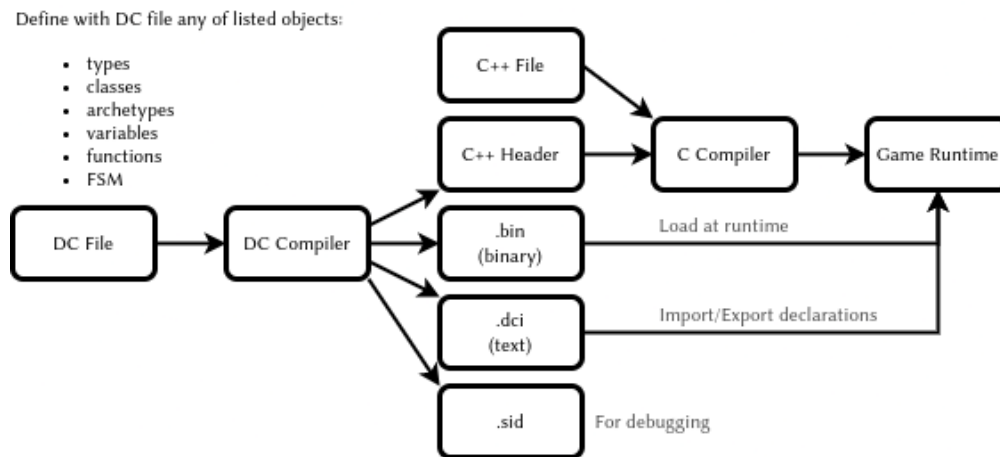


Рис. 5: Редактирование модели мира [Gre17]

5 Spawning

Основа гибкого создания новых экземпляров игровых сущностей — *spawning* — это использование скрипт-процессов. Такие процессы делают всю необходимую работу по созданию объектов, а затем производят инъекцию нужных данных. Система *spawning* использует *фабрику объектов*, которая имеет таблицу имен типов и архетипов и хранит информацию о наследовании и размере классов. Также система должна уметь перемещать объекты для эффективного использования памяти.

Система может запрашивать максимальный объём памяти, необходимый для хранения объекта, а после инстанцирования освобождать неиспользованный фрагмент памяти. После релоцирования, все объекты в памяти должны располагаться оптимальным образом.

Более качественный результат может дать *Data Oriented* подход, который использует пулы (pools) гомогенных объектов.

В целом, *spawning*-система должна решать следующие проблемы:

- Следить за уникальностью идентификаторов
- Создавать всю необходимую иерархию объектов
- Настраивать все необходимые зависимости
- Эффективно использовать память
- Бережно использовать ресурсы процессора. Например, создавать объект за несколько шагов: запрос, создание, инициализация, запрос создания дочерних и т.д.
- Уметь использовать при создании объектов очередь с приоритетами *priority queue*
- Позволять создавать объекты различными способами:
 - *Spawner* — объектом в мире с параметрами инъекции
 - *C* — кодом с аргументами инъекции
 - *Script* — кодом с аргументами инъекции
 - *Cloning* — копированием объекта
 - *Replication* — копированием по сети

6 Сигналы и сообщения

Цель программиста создавать код с минимумом зависимостей. Два взаимодействующих объекта не должны знать друг о друге слишком много. Вместо этого, используя полиморфизм, они должны говорить друг с другом на абстрактном языке сообщений. Для этого хорошо подходят контейнеры данных, где для ключей *key* имеется запись данных *value* типа *variant*. При этом, в качестве указателей на объекты лучше использовать имена *StringId* или идентификаторы *Handler*.

7 State Update

Обновление всех объектов происходит через *batched*- и *bucket*-метод.

- *batched* — обновление всех компонентов одного типа *Data Oriented Programming*
- *bucket* — обновление объектов по приоритетам, для устранения проблемы взаимозависимости

Ниже приведен пример *batched* и *backed* обновления [Gre17].

```
while (true)
{
    PollJoypad();
    float dt = GetFrameDeltaTime();
    // Backed update game objects
    for (each bucket)
    {
        for (each gameObject in bucket)
        {
            gameObject.Update(dt);
        }
    }
    // Batched update components
    g_animationEngine.Update(dt);
    g_physicsEngine.Simulate(dt);
    g_collisionEngine.Run(dt);
    g_audioEngine.Update(dt);
    g_renderingEngine.RenderFrame();
    g_videoDriver.FlipBuffers();
}
```

Обновление по фазам устраняет проблемы взаимозависимости. Суть решения в том, чтобы обновлять объекты не за один, а за несколько проходов. В своих проектах я использую именно такой способ обновления объектов. Смотри пример ниже [Gre17]. В принципе, количество фаз может быть любым, но в моей практике использовались только две.

```
while (true) // main game loop
{
    // ...
    for (each gameObject)
        gameObject.PreAnimUpdate(dt);

    g_animationEngine.CalculateIntermediatePoses(dt);

    for (each gameObject)
        gameObject.PostAnimUpdate(dt);

    g_ragdollSystem.ApplySkeletonsToRagDolls();
    g_physicsEngine.Simulate(dt);
    g_collisionEngine.DetectAndResolveCollisions(dt);
    g_ragdollSystem.ApplyRagDollsToSkeletons();
    g_animationEngine.FinalizePoseAndMatrixPalette();

    for (each gameObject)
        gameObject.FinalUpdate(dt);
    // ...
}
```

То, как необходимо обновлять объекты, в большой степени зависит от самой иг-

ры. И решение должно приниматься в каждом конкретном случае, для каждого конкретного проекта.

8 Идентификаторы объектов

Все имена объектов трансформируются в целочисленные значения (*integer*) с помощью алгоритма *CRC32*. В исходном коде на *C* используется макрос *SID(s)*, который перед компиляцией конвертируется в *SID(n, s)*. Идентификаторы всех строк собираются в отдельном текстовом *.sid*-файле для отладки. Пример макроса в исходном *C*-файле приведен ниже.

```
#define SID(n,...) n
```

В результате во время компиляции исходные строки полностью отбрасываются, но попадают в *.sid*-файл. Сгенерированные *.cpp/.h*-файлы имеют ссылку на оригинальный файл. Пример такой ссылки показан ниже.

```
#line 1 "original_file.cpp"
```

В целом процесс выглядит так, как показано на диаграмме 6.

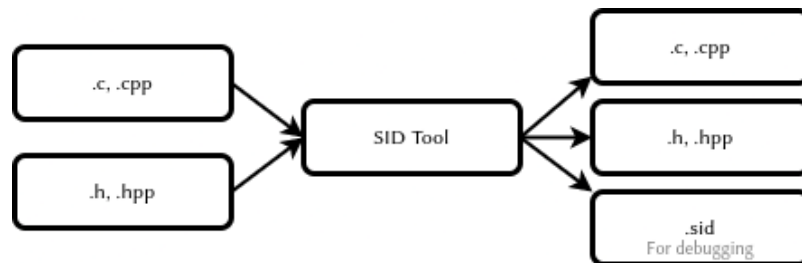


Рис. 6: Обработка *C* файлов перед компиляцией

Современная версия *C* позволяет использовать для этой цели *constexpr* для генерации *StringId*. Пример такой функции приведен ниже:

```
// Usage: find_character("player"_id)
constexpr StringId operator "" _id(const char* v, unsigned int c) {
    return crc32_helper(v, c, 0xFFFFFFFF);
}
```

Пример генератора *StringId* приведен ниже. Эта функция работает в коде дизассемблера *.bin*-файлов игры.

```

# Python
def create_table(poly):
    init=0
    l=[0]*256
    for i in range(256):
        t=init^(i<<24)
        for j in range(8):
            mask=1<<31
            if(mask&t!=0):
                t=(t<<1)^poly
            else:
                t=(t<<1)
        l[i]=t&0xffffffff
    return l

crc32_table = create_table(0x04c11db7)

def crc32(s, init=0):
    crc = init
    if s:
        for c in s:
            crc = (crc32_table[ ((crc>>24) ^ ord(c)) & 0xff ] \
                    ^ (crc << 8)) & 0xffffffff
    return crc

```

9 Синтаксис DC

Язык позволяет декларировать новые типы, ниже приведен пример четырехкомпонентного вектора [Lie08].

```
(deftype vec4 (:align 16)
  ((x float)
   (y float)
   (z float)
   (w float :default 0)
  )
)
```

При декларировании можно использовать наследование. Примеры такого наследования приведены ниже [Lie08].

```
(deftype quaternion (:parent vec4)
  ())

(deftype point (:parent vec4)
  ((w float :default 1)
   ))
```

Еще один пример, но теперь композиции классов [Lie08].

```
(deftype locator ()
  ((trans point :inline #t)
   (rot quaternion :inline #t)
  )
)
```

В результате, *DC*-компилятор преобразует структуру в содержимое *.h*-файла [Lie08].

```
struct Locator
{
    Point m_trans;
    Quaternion m_rot;
};
```

Декларировать можно не только типы, но и функции. Ниже приведен пример функции *axis-angle->quat* [Lie08].

```
(define (axis-angle->quat axis angle)
  (let ((sin-angle/2 (sin (* 0.5 angle))))
    (new quaternion
      :x (* (-> axis x) sin-angle/2)
      :y (* (-> axis y) sin-angle/2)
      :z (* (-> axis z) sin-angle/2)
      :w (cos (* 0.5 angle))
    )))
```

Одна из важных особенностей *LISP*-подобного языка — это возможность создания предметно-ориентированных языков *DSL*. Это позволяет писать код и декларировать данные более компактно, без лишних церемоний [Lie08].

```
(define *y-axis* (new vec4 :x 0 :y 1 :z 0))
(define *origin* (new point :x 0 :y 0 :z 0))
```


При этом, объявления данных могут использовать функции. Пример определения стартовой точки игрока показан ниже. Здесь в качестве значения угла поворота используется функция, вычисляющая *quaternion* из *угла* и *оси вращения* [Lie08].

```
(define-export *player-start*  
  (new locator  
    :trans *origin*  
    :rot (axis-angle->quaternion *y-axis* 45)  
  ))
```

Использование дефиниций в *DC* файле из *C* исходного кода выглядит как на примере ниже [Gre14].

```
#include "dc-types.h"  
  
const Locator * pLoc = DcLookupSymbol("*player-start*");  
Point pos = pLoc->m_trans;
```

10 Анимационные состояния

Анимационные состояния реализуются как структуры данных. Требуется наличие соответствующего *C*-кода для того, чтобы интерпретировать эти состояния и произвести конструирование необходимых объектов в памяти системы. Ниже приведено простое анимационное состояние *pirate-jump* [Gre17].

```
(define-state simple
  :name      "pirate-jump"
  :clip      "pirate-jump"
  :flags     (anim-state-flag no-adjust-to-ground)
)
```

Пример состояния сложной анимации приведен ниже [Gre17]. В данном случае производится линейная интерполяция двух анимаций: *pirate-jump* и *pirate-scare*.

```
(define-state complex
  :name      "pirate-jump"
  :tree
  (anim-node-lerp
    (anim-node-clip "pirate-jump")
    (anim-node-clip "pirate-scare")
  )
)
```

Еще один пример приведен ниже, в нем присутствует дерево различных узлов (node), которые производят операции смешивания анимаций [Gre17].

```
(define-state complex
  :name      "pirate-jump"
  :tree
  (anim-node-lerp
    (anim-node-additive
      (anim-node-additive
        (anim-node-clip "pirate-jump-f")
        (anim-node-clip "pirate-scare-f")
      )
      (anim-node-clip "pirate-felldown-f")
    )
    (anim-node-additive
      (anim-node-additive
        (anim-node-clip "pirate-jump-b")
        (anim-node-clip "pirate-scare-b")
      )
      (anim-node-clip "pirate-felldown-b")
    )
  )
)
```

Описание анимационных переходов между состояниями приведена ниже [Gre17].

```

;; nb aim-tree is the macro definition
(define-state complex
  :name "s-turret-idle"
  :tree (aim-tree (anim-node-clip "turret-aim-all-base")
                  "turret-aim-all-left-right"
                  "turret-aim-all-left-updown")
  :transitions (
    (transition "reload" "s_turret-reload"
               (range - -) :fade-time 0.2)

    (transition "step-left" "s_turret-step-left"
               (range - -) :fade-time 0.2)

    (transition "step-right" "s_turret-ste-right"
               (range - -) :fade-time 0.2)

    (transition "reload" "s_turret-fire"
               (range - -) :fade-time 0.1)

    ;; invoke previously defined group of transitions
    ;; it is used when the same set of transitions needed
    ;; to be used in the other state
    (transition-group "combat-gunpout-idle-mode")

    ;; specifies a transition that is
    ;; taken upon reaching the end of the state's
    ;; local time line if no other transition
    ;; has been taken before then
    (transition-end "s-turret-idle")
  )
)

```

Подобным образом можно кодировать и другие системы игры например *AI*, *Melee* [Cho21], и другие.

11 Конечные автоматы

В компании *ND* под состоянием понимается определенный набор процессов, которые выполняются для конкретного хост-объекта или как самостоятельные процессы в памяти [Gre06]. Пример конечного автомата [Gre06] анимированной сцены показан ниже.

```
;; Сцена с аварией автобуса
(define-state-script ("wz-bus-crash")
  ;; состояние spawn солдат
  (state ("spawn-soldiers")
    (on (begin)
      ;; отключить управление игроком, но кроме правой кнопки
      [player-disable-controls
       (controls all-but-right-stick)]
      ;; создать солдат
      [spawn-npc-in-combat "npc-wz-52"]
      [spawn-npc-in-combat "npc-wz-53"]
      ...
      ;; перейти в состояние crash
      [go "crash"]
    )
  )
  ...
)
```

12 Объявление переменных состояния

Состояние может иметь собственные переменные для сохранения различных значений или обмена данными.

```
;; Сцена с аварией автобуса
(define-state-script ("kickable-gate")
  :initial-state "closed"
  :declarations (decl-list
    (var "num-attempts" :type int32)
    (var "is-locked" ::default #t)))
  ...
)
```

13 Параллельное исполнение процессов

Каждое состояние объекта можно представить как множество параллельных треков. Некоторые из которых выполняются от начала до конца в каждом кадре, а другие приостанавливаются и продолжают выполнение в ответ на определенное событие. Существуют еще и отдельные треки, которые запускаются по событию. В начале исполняется код инициализации, а в конце — код финализации. Диаграмма 7 иллюстрирует одно состояние.

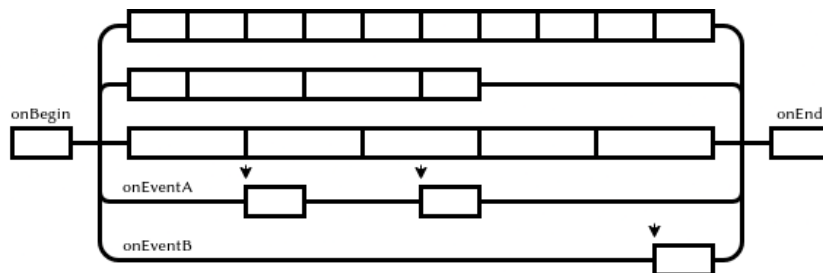


Рис. 7: Треки одного состояния

Следующее состояние [Gre06] запускает четыре трека, то есть четыре параллельных процесса. Каждый процесс обрабатывает свой сценарий и отправляет сообщение в финале. Каждый процесс может приостанавливаться в ожидании другого процесса или в ожидании определенного сценария.

```
(state ("crash")
  (on (begin)
    ;; процесс анимации автобуса
    (track ("bus")
      [wait-animate "bus-1" "bus-crash"
       [get-locator "ref-bus-crash-1"]]
      [signal "bus-done"]
    )
    ;; процесс анимации игрока
    (track ("player")
      [animate "player" "player-watch-crash"
       [get-locator "ref-bus-crash-1"]]
      [wait-until-frame 250]
      [say "player" "vox-wz-drk-01-what-the"]
      [signal "drake-done"]
    )
    ;; процесс анимации того, кого собьет автобус
    (track ("guy-hit-by-bus")
      [wait-animate "npc-wz-52" "npc-hit-by-bus"
       [get-locator "ref-bus-crash-1"]]
      [npc-die "npc-wz-52"]
      [signal "npc-dead"]
    )
    ;; процесс ожидания всех остальных процессов
    (track ("wait-for-all-done")
      [wait-for-signal "bus-done"]
      [wait-for-signal "drake-done"]
      [wait-for-signal "npc-dead"]
      [go "done"]
    )
  )
  ...
)
```

В конечном итоге, система треков многослойна, при этом верхние уровни контролируют нижние. Примеры уровней от верхнего к нижнему приведены ниже:

- Процессы верхнего уровня игры, а также глобальные процессы, например, смена дня и ночи
- Процессы текущего мира
- Процессы текущей зоны

- Процессы батл-зоны
- Процессы группового интеллекта
- Процессы персонажей
- Процессы дочерних объектов

Этот подход весьма элегантный и простой, в котором есть некоторые недостатки, например:

- Дизайнеры должны уметь программировать на языке скриптов
- Процесс исполнения кода в треках не зависит от времени, то есть не может исполняться в обратном порядке или совершать скачки во времени.

Впрочем, последнее бывает возможно в *Data-Driven* системах.

14 Интеграция VM в Engine

Исходный код *DC*-файлов компилируется в байткод. Более детально этот аспект будет затронут ниже. При любом способе интеграции динамического языка в систему, требуется механизм этой интеграции *Reflection*, *FFI*, и т.д.

В компании *ND* применен очень простой, но весьма эффективный способ интеграции виртуальной машины и самого движка. Для этого используется хэш-таблица, в которой к каждому ключу *ssid* имеется указатель на *C*-функцию. Это функция с переменным числом аргументов, которые при этом имеют *variant*-тип. Количество возможных типов аргументов весьма невелико: *integer*, *float*, *StringId*, *Pointer*.

Пример такой функции приведен ниже [Gre06]. Для доступа к объектам сцены используются имена объектов в виде *StringId*, при этом, зарезервированное имя *self* адресует хост-объект процесса.

```
Variant ScriptWaitAnimate(int argc, Variant* argv)
{
    StringId objName = SC_ARG(0,StringId, NULL);
    StringId animName = SC_ARG(1,StringId, NULL);

    if(!objName)
        // The ScriptError is a function return Variant(false)
        // And print the error message
        return ScriptError(
            "wait-animate: expected object name (arg1)\n");
    if(!animName)
        return ScriptError(
            "wait-animate: expect animation name (arg2)\n");

    // find the object
    ProcessGameObject* pObj = g_processMgr.Lookup(objName);

    if(!pObj)
        return ScriptError("wait-animate: could not found %s\n",
            StringIdToString(objName));

    // instruct object to play animation, and wakeup
    // this script when done
    pObj->WaitAnimate(animName, g_scriptContext);
    g_scriptContext.Suspend(); // go to sleep until animation complete
    return Variant(true);
}
```

Теперь *C*-функцию *ScriptWaitAnimate* можно декларировать в динамической среде программирования, смотри пример ниже [Gre06]. Декларация нужна лишь для объявления сигнатуры метода, то есть для проверки типов.

```
(define-c-function wait-animate
  (object-name string)
  (anim-name string)
)
```

15 DC компилятор

Реализован на *Racket*, хотя мог быть реализован и на *C*, *Go* или любом другом языке. Использование *Racket* может быть связано со следующими причинами:

- Это среда, специально нацеленная на разработку *DSL*
- Компилятор *Racket* поддерживает большое количество уже реализованных для платформы языков, таких как *Typed Racket*
- Это зрелый продукт, хорошо зарекомендовавший себя в академической среде
- У *Racket* есть собственная *IDE* — *DrRacket*. Она проста в установке и использовании, и при этом предоставляемые ею средства весьма наглядны и информативны.
- Продвинутые программисты могут использовать другой редактор, например *EMACS*

16 Формат SID-файла

Наличие такого файла — это мое предположение. Файл имеет текстовый формат и предназначен для хранения текстовых форм каждого *StringId*. Это может быть полезно при отладке программ. Ниже приведен пример фрагмента этого файла.

```
dbd3d0d8 is-test-task?  
2a990f91 is-demo-part-2?  
bff578ab is-t2?  
dcf596c6 get-difficulty  
a86d881d get-dda
```

17 Формат DCI-файла

Данный файл предназначен для линковки модулей. В каждом файле в текстовой форме приводятся все импортируемые файлы и экспортируемые определения. Теоретически, в режиме отладки, в файл могут быть помещены все текстовые формы.

```
;; script-user-funcs.dci
(script-user-funcs (69857) ;
  ;; Import files
  (import script-funcs vox-defines
    vox-remap-defines fact-defines
    vox-action-defines)
  ;; Export symbols
  (export disable-relocation add-int32 subtract-int32 string)
)
```

18 Формат бинарного DC-файла

Бинарный файл не документирован и ещё не полностью исследован, но некоторые выводы уже можно сделать. Формат файла очень прост и дружелюбен для среды исполнения. Каждый файл начинается с 32 байт заголовка.

```
struct DcHeader {
    char magic[4] = "DC00"; // Магическая сигнатура
    u32 unknown1;
    u32 relocation1;
    u32 unknown2;
    u32 unknown3;
    u32 definitions_count; // Количество дефиниций в файле
    u32 definitions_offset; // Начало данных с дефинициями
    u32 unknown4; // PS4 version only
}
```

Каждое определение имеет имя, тип и смещение от начала файла. Примечательно, что тип объекта записан в текстовой форме, конвертированной в *StringId*.

```
class DcDefinition {
    u32 nameId; // SID aka StringId("player")
    u32 typeId; // SID aka StringId("lambda")
    u32 offset; // Start of the descriptor
    u32 unknown1; // PS4 version only
}
```

Ниже перечислены, предположительно, основные типы определений.

```
vector      = 0x012f77fe
string      = 0x0b3952e7
float       = 0x0f182ec3
angle       = 0x13812cd6
state       = 0x2e6743e3
direction   = 0x7194cbe7
color       = 0x71e73c6c
boolean     = 0x8b4e76ff
vec4        = 0x93bd2e95
script-lambda = 0x9ed499e1
function    = 0xab3eb31f
int32       = 0xc7cb2752
```

Исходный код транслируется в тип *script-lambda* или *function*. Дефиниция указывает на дескриптор. Дескриптор имеет указатель на блоки кода и данных *lambda* функции.

```
struct DcDescriptor {
    u32 code; // Смещение начала кода
    u32 unknown1;
    u32 data; // Смещение начала данных
    u32 unknown2;
}
```

19 Виртуальная машина

Виртуальная машина имеет список состояний процессов, в котором расположены указатели на блок памяти, хранящий *окружение* (*environment*). В окружении хранятся:

- Указатель на исполняемую *lambda* функцию
- Индекс текущей инструкции
- Указатель на родительское окружение, если окружения не организованы в виде стека
- Блок регистров, каждый из которых имеет вариантный тип

Структура окружения *VM* изображена на рисунке 8

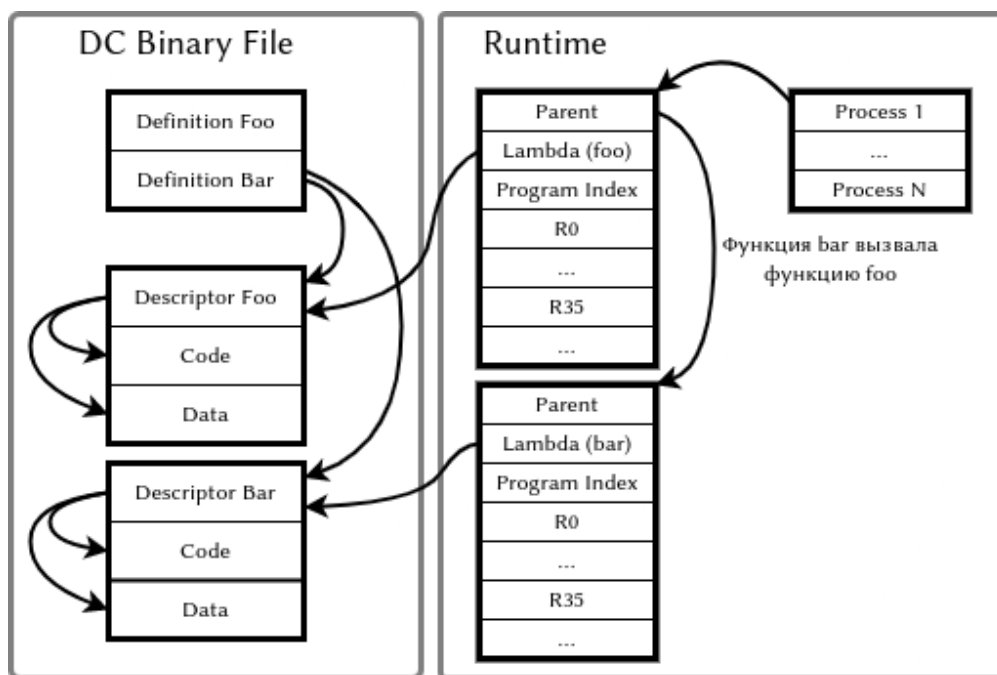


Рис. 8: Runtime виртуальной машины [Gre06]

20 Система команд VM

Код состоит из гомогенных команд в виде массива из 32-х битных значений. Кроме кода операции, имеется три операнда a, b, c . Для некоторых команд операнд c используется как непосредственное значение k , для других операнды b и c объединяются в 16-ти битное значение kk .

```
struct DcInstruction {  
    u8 opcode; // Opcode  
    u8 a;      // Register number  
    u8 b;      // Register number  
    u8 c;      // Register number  
}
```

Доступ к константам идет по адресу данных из дескриптора. Значение регистра, умноженное на N^4 используется как смещение области данных, где хранятся:

- Целые числа $I8, U8, I16, U16, I32, U32, I64, U64$. Однако, в исследованной мною игре использовались только $I32$ и $U32$
- 32-битные числа с плавающей точкой
- Строки текста

В таблице ниже приведена система команд виртуальной машины. В колонке $Q\text{-}ty$ дается приблизительное количество использований команды в игре.

⁴⁸ для *The Last of Us PS4* и 4 для *The Last of Us PS3*

Таблица 1: Команды виртуальной машины с 0x00 по 0x1F

Opcode	Name	Q-ty	Comment
0x00	return	1412	return aRes, b (allways equal a)
0x01	intAdd	130	$a = b + c$
0x02	intSub	19	$a = b - c$
0x03	intMul	1	$a = b * c$
0x04	intDiv	0	$a = b / c$
0x05	floatAdd	32	$a = b + c$
0x06	floatSub	44	$a = b - c$
0x07	floatMul	68	$a = b * c$
0x08	floatDiv	30	$a = b / c$
0x09	loadStaticInt	0	$a = (\text{int})\text{data}[\text{kk} * \text{N}]$
0x0A	loadStaticFloat	0	$a = (\text{float})\text{data}[\text{kk} * \text{N}]$
0x0B	loadStaticPointer	0	$a = (\text{char}^*)\text{data}[\text{kk} * \text{N}]$
0x0C	loadImm	3577	$a = \text{BC}$
0x0D	loadInt	78	$a = (\text{int}) * b$
0x0E	loadFloat	129	$a = (\text{float}) * b$
0x0F	loadPointer	2	$a = (\text{pointer}) * b$
0x10	storeInt	0	$(\text{int}^*)a = b$
0x11	storeFloat	0	$(\text{float}^*)a = b$
0x12	storePointer	0	$(\text{char}^{**})a = b$
0x13	lookupInt	0	$a = (\text{int})\text{lookup}((\text{sid})\text{data}[\text{kk} * \text{N}])$
0x14	lookupFloat	0	$a = (\text{float})\text{lookuo}((\text{sid})\text{data}[\text{kk} * \text{N}])$
0x15	lookupPointer	8313	$a = (\text{char}^*)\text{lookup}((\text{sid})\text{data}[\text{kk} * \text{N}])$
0x16	moveInt	0	$a = b$
0x17	moveFloat	0	$a = b$
0x18	movePointer	0	$a = b$
0x19	castInteger	23	$a = (\text{int})b$
0x1A	castFloat	86	$a = (\text{float})b$
0x1B	call	1429	Call script function(aArg, bRes, argc)
0x1C	callFf	6866	Call native function(aArg, bRes, argc)
0x1D	cmpEqual	721	$a = b == c$
0x1E	cmpGt	49	$a = b > c$
0x1F	cmpGtEqual	20	$a = b \geq c$

Таблица 2: Команды виртуальной машины с 0x20 по 0x3F

Opcode	Name	Q-ty	Comment
0x20	cmpLt	141	$a = b < c$
0x21	cmpLtEqual	16	$a = b \leq c$
0x22	cmpFloatEqual	19	$a = b == c$
0x23	cmpFloatGt	108	$a = b > c$
0x24	cmpFloatGtEqual	31	$a = b \geq c$
0x25	cmpFloatLt	153	$a = b < c$
0x26	cmpFloatLtEqual	44	$a = b \leq c$
0x27	intMod	2	$a = \text{mod}(b)$
0x28	floatMod	0	$a = \text{fmod}(b)$
0x29	intAbs	0	$a = \text{abs}(b)$
0x2A	floatAbs	18	$a = \text{fabs}(b)$
0x2B	(not available)	0	
0x2C	(not available)	0	
0x2D	branch	844	rjump kk
0x2E	branchIf	348	if (a) rjmp kk
0x2F	branchIfNot	2063	if (not a) rjmp kk
0x30	opLogNot	417	$a = \text{not } b$
0x31	opBitAnd	1	$a = b \text{ and } c$
0x32	opBitNot	0	$a = \text{bnot}(b)$
0x33	opBitOr	0	$a = b \text{ bor } c$
0x34	opBitXor	0	$a = b \text{ bxor } c$
0x35	opBitNor	0	$a = \text{bont } (b \text{ bor } c)$
0x36	opLogAnd	0	$a = b \text{ and } c$
0x37	opLogOr	0	$a = a \text{ or } c$
0x38	intNeg	0	$a = -b$
0x39	floatNeg	0	$a = -b$
0x3A	loadParmCnt	1	$a = \text{argc}$
0x3B	intAddImm	158	$a = b + k$
0x3C	intSubImm	0	$a = b - k$
0x3D	intMulImm	0	$a = b * k$
0x3E	intDivImm	0	$a = b / k$
0x3F	loadStaticI32Imm	7128	$a = (\text{i32})\text{data}[\text{kk} * \text{N}]$

Таблица 3: Команды виртуальной машины с 0x40 по 0x4A

Opcode	Name	Q-ty	Comment
0x40	loadStaticFloat	1699	a = (float)data[kk*N])
0x41	loadStaticPointer	558	a = (char*)&data[data[kk*N]]
0x42	intAsh	0	a shift b bits left/right
0x43	move	27681	a = b
0x44	loadStaticU32	0	a = (u32)data[kk*N])
0x45	loadStaticI8	0	a = (i8)data[kk*N])
0x46	loadStaticU8	0	a = (u8)data[kk*N])
0x47	loadStaticI16	0	a = (i16)data[kk*N])
0x48	loadStaticU16	0	a = (u16)data[kk*N])
0x49	loadStaticI64	0	a = (i64)data[kk*N])
0x4A	loadStaticU64	0	a = (u64)data[kk*N])

20.1 Использование регистров и констант

Приведу несколько примеров. В игре *The Last of Us* не было ни одного доступа к регистру, большему чем *R34*. При этом, регистры *R24* и выше использовались, как правило, для передачи аргументов.

Код функции *npc-smart-move-to* выглядит так:

```
npc-smart-move-to:
; Get 4 arguments
move    r0, r24          ; r0 = r24
move    r1, r25          ; r1 = r25
move    r2, r26          ; r2 = r26
move    r3, r27          ; r3 = r27
; Find object reference
lookupPointer r4, data[0] ; r4 = StringId(0xa93d2926)
move    r5, r1           ; r5 = r1
; Set arguments
move    r24, r5           ; r24 = r5
; Call function
callFf  r4,r4,1
; Use the result of functo
branchIfNot r4, 0x00001770 ; ix00001770
```

Ниже приведен исходный код функции *vector-scale*. Можно заметить, что компилятор не имеет средств для качественной оптимизации. Но это не является большой проблемой, так как игра имеет хорошую архитектуру и четкое разделение между процессами высокой интенсивности и логикой игры, выполненной на *VM*.

```
vector-scale(scalar, vector*)
move    r0, r24          ; r0 = r24 = scalar value
move    r1, r25          ; r1 = r25 = vector pointer
; Get native function pointer
lookupPointer r2, data[0] ; r2 = StringId(0xcd4b9c1b)
; Scale X
move    r3, r0           ; r3 = r0
move    r4, r1           ; r4 = r1 = @vector.x
loadFloat r4, (r4)       ; r4 = *r4 = vector.x
floatMul r3, r3, r4      ; r3 = r3 * r4 = x * scale
; Scale Y
move    r4, r0           ; r4 = r0 = scalar
move    r5, r1           ; r5 = r1 = @vector
intAddImm r5, r5, 0x04; r5 = r5 + 4 = @vector.y
loadFloat r5, (r5)       ; r5 = *r5 = vector.y
floatMul r4, r4, r5      ; r4 = r4 * r5 = y * scale
; Scale Z
move    r5, r0           ; r5 = r0 = scalar
move    r6, r1           ; r6 = r1 = @vector
intAddImm r6, r6, 0x08; r6 = r6 + 8 = @vector.z
loadFloat r6, (r6)       ; r6 = *r6 = vector.z
floatMul r5, r5, r6      ; r5 = r5 * r6 = vector.z * scale
loadStaticFloat r6, data[1] ; r6 = 1
; Call function with 4 values
move    r24, r3          ; r24 = r3 = x
move    r25, r4          ; r25 = r4 = y
move    r26, r5          ; r26 = r5 = z
move    r27, r6          ; r27 = r6 = w = 1
callFf  r2, r2, 4        ; function(x,y,z,w)
return  r2, r2
```

Передача сообщения объекту присутствует в коде ниже.

```

kill-rigid-body(x,y)
  move      r0, r24      ; r0 = r24
  move      r1, r25      ; r1 = r25
  lookup    r2, data[0] ; r2 = StringId(NATIVE:send-event)
  loadStaticI32Imm r3, data[1]; r3 = 274190794 (0x1057d1ca)
  move      r4, r0      ; r4 = r0
  loadImm   r5, 0x0004   ; r5 = 4
  move      r6, r1      ; r6 = r1
  move      r24, r3      ; r24 = r3
  move      r25, r4      ; r25 = r4
  move      r26, r5      ; r26 = r5
  move      r27, r6      ; r27 = r6
  callFf    r2, r2, 4    ; send-event(264190794,x,4,y)
  return    r2,r2

```

Набор допустимых типов *variant* контейнера не включает *StringId* вместо этого используется *integer* значение. Это можно видеть на примере ниже.

```

cloth-remove-external-collider(arg)
  move      r0, r24      ; r0 = r24
  lookupPointer r1, data[0] ; r1 = StringId(send-event)
  ; Load string id 'cloth-remove-external-collider'
  ; as integer 32 bits constant 1119424146
  loadStaticI32Imm r2, data[1] ; r2 = 1119424146
  move      r3, r0      ; r3 = r0
  move      r24, r2      ; r24 = r2
  move      r25, r3      ; r25 = r3
  ; callFf(sid(cloth-remove-external-collider), arg)
  callFf    r1, r1, 2
  return    r1, r1

```

Типы данных размер которых превышает *variant* контейнер передаются как указатель на объект или как хэндрер объекта в пуле.

21 Результат

Безусловно, данное введение не велико и многое упрощает, описывая лишь базу игровой системы и пайплайна в целом. Однако, из этого описания можно составить представление о наиболее важных решениях:

- Избегать проектирования инструментов с перегруженным GUI, лучше использовать потратить силы на саму игру
- Распределенная система управления ассетами должна быть пригодна для совместной работы и должна поддерживать множество проектов и их вариантов
- Ядро игры должно быть производительным, а инструменты дизайнера — гибкими
- Использование *LISP*-подобных языков позволяет удобно описывать данные и код в одном файле

Весь игровой процесс состоит из тысяч параллельных процессов, каждый из которых подобен сценарию фильма. Я наблюдала, что в коде присутствуют функции, код которых занимает несколько экранов.

В целом, большинство упомянутых подходов, применимо и к другим средам разработки игр, таким как *Unity 3D* и *Unreal Engine*. У меня есть завершенные коммерческие проекты на *Unity 3D*, сделанные с использованием принципов, изложенных в этом документе. Я оцениваю этот результат как весьма успешный.

22 В завершение

Стоит сказать, что проект *The Last of Us on the Playstation 3* потребовал следующих ресурсов:

- 16 программистов
 - из них лишь два *tool* программиста
- 20 дизайнеров
- 120 аниматоров
- 6000 DC файлов
 - 120 Мб суммарный объём DC-исходных файлов
 - 45 Мб суммарный объём DC-бинарных файлов
 - Динамически загружаемые в пространство 5 Мб в области хипа

Полагаю, здесь есть о чем подумать.

Список литературы

- [Cho21] Ming-Lun “Allen” Chou. Melee ai in "the last of us part ii". 2021.
- [Coh10] Terrence Cohen. A dynamic component architecture for high performance gameplay. 2010.
- [Gre06] Jason Gregory. State based scripting in "uncharted: Drake’s fortune". 2006.
- [Gre14] Jason Gregory. Game engine architecture. 2014.
- [Gre17] Jason Gregory. Game object model and scripting. 2017.
- [Lie08] Dan Liebgold. Adventures in data compilation in "uncharted: Drake’s fortune". 2008.